

LAB PROGRAMS

SET[23-30]

Name:E.Kavya

Reg NO:192424365

Date:10/8/26

Set 23: Employee Performance Analysis

Dataset: Employee Performance

Employee ID	Department	Years of Service	Performance Score
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78
4	Sales	4	90
5	HR	2	76

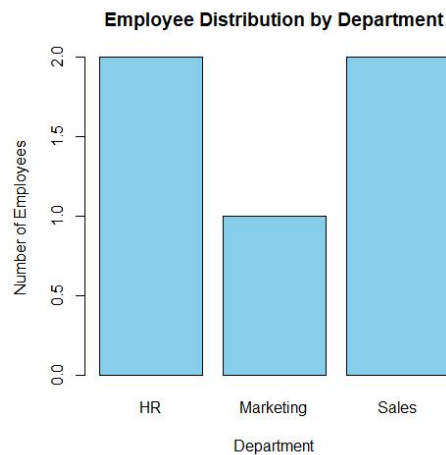
1. In R, create a bar chart to visualize the distribution of employees across different departments.

Label the chart elements.

Code:

```
employee_id <- c(1, 2, 3, 4, 5)
department <- c("Sales", "HR", "Marketing", "Sales", "HR")
years_service <- c(5, 3, 7, 4, 2)
performance_score <- c(85, 92, 78, 90, 76)
employee_data <- data.frame(
  Employee_ID = employee_id,
  Department = department,
  Years_of_Service = years_service,
  Performance_Score = performance_score
)
dept_count <- table(employee_data$Department)
barplot(dept_count,
  main = "Employee Distribution by Department",
  xlab = "Department",
  ylab = "Number of Employees",
  col = "skyblue")
text(x = seq_along(dept_count),
  y = dept_count,
  labels = dept_count,
  pos = 3)
```

Output:

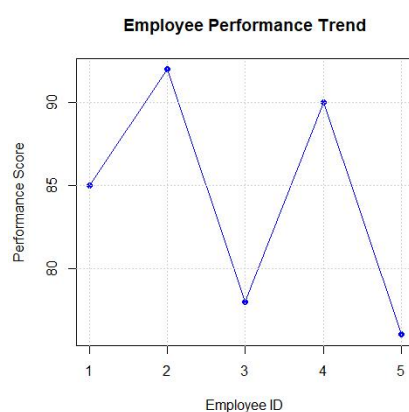


2. In R, generate a line chart to visualize the performance trend of employees over time. Label the axes and title the chart.

Code:

```
plot(employee_data$Employee_ID,
      employee_data$Performance_Score,
      type = "o",
      pch = 16,
      col = "blue",
      main = "Employee Performance Trend",
      xlab = "Employee ID",
      ylab = "Performance Score",
      xaxt = "n")
axis(1,
     at = employee_data$Employee_ID,
     labels = employee_data$Employee_ID)
grid()
```

Output:



3. In Tableau, build a dashboard combining a bar chart showing department distribution and a scatter plot displaying the relationship between years of service and performance scores.

Code:

```
if (!require(ggplot2)) install.packages("ggplot2")
if (!require(patchwork)) install.packages("patchwork")
library(ggplot2)
```

```

library(patchwork)
employee_data <- data.frame(
  Employee_ID = c(1, 2, 3, 4, 5),
  Department = c("Sales", "HR", "Marketing", "Sales", "HR"),
  Years_of_Service = c(5, 3, 7, 4, 2),
  Performance_Score = c(85, 92, 78, 90, 76)
)
bar_chart <- ggplot(employee_data, aes(x = Department)) +
  geom_bar(fill = "skyblue") +
  geom_text(
    stat = "count",
    aes(label = after_stat(count)),
    vjust = -0.5
  ) +
  labs(
    title = "Employee Distribution by Department",
    x = "Department",
    y = "Number of Employees"
  ) +
  theme_minimal()
scatter_chart <- ggplot(
  employee_data,
  aes(x = Years_of_Service,
    y = Performance_Score,
    color = Department)
) +
  geom_point(size = 4) +
  geom_text(aes(label = Employee_ID),
    vjust = -1,
    size = 4) +
  labs(
    title = "Years of Service vs Performance Score",
    x = "Years of Service",
    y = "Performance Score",
    color = "Department"
  ) +
  theme_minimal()
bar_chart + scatter_chart

```

Output:



4. In R, develop a table showing the performance data for each employee. Label the table elements.

Code:

```
performance_table <- data.frame(  
  Employee_ID = c(1, 2, 3, 4, 5),  
  Department = c("Sales", "HR", "Marketing", "Sales", "HR"),  
  Years_of_Service = c(5, 3, 7, 4, 2),  
  Performance_Score = c(85, 92, 78, 90, 76)  
)  
print(performance_table)
```

Output:

	Employee_ID	Department	Years_of_Service	Performance_Score
1	1	Sales	5	85
2	2	HR	3	92
3	3	Marketing	7	78
4	4	Sales	4	90
5	5	HR	2	76

Set 24 Product Inventory Management

Dataset: Product Inventory

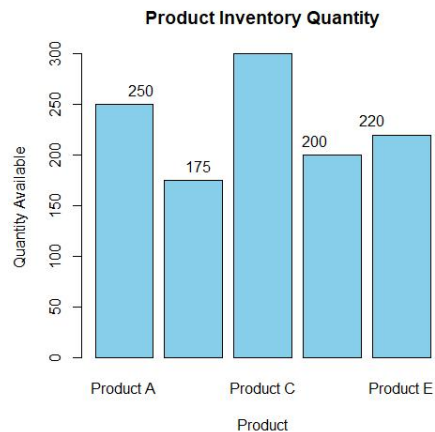
	Product_ID	Product_Name	Quantity_Available
1	Product A	250	
2	Product B	175	
3	Product C	300	
4	Product D	200	
5	Product E	220	

1. In R, create a bar chart to visualize the quantity of each product in the inventory. Label the axes and title the chart.

Code:

```
product_id <- c(1, 2, 3, 4, 5)  
product_name <- c("Product A", "Product B", "Product C", "Product D", "Product E")  
quantity <- c(250, 175, 300, 200, 220)  
inventory <- data.frame(  
  Product_ID = product_id,  
  Product_Name = product_name,  
  Quantity_Available = quantity  
)  
barplot(inventory$Quantity_Available,  
  names.arg = inventory$Product_Name,  
  col = "skyblue",  
  main = "Product Inventory Quantity",  
  xlab = "Product",  
  ylab = "Quantity Available")  
text(x = seq_along(quantity),  
  y = quantity,  
  labels = quantity,  
  pos = 3)
```

Output:

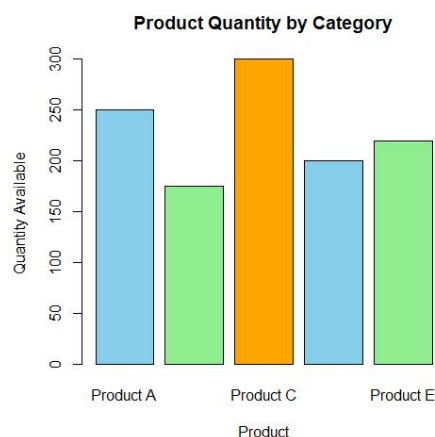


2. In R, generate a stacked bar chart to show the quantity of each product within different product categories.

Code:

```
category <- c("Electronics",
              "Electronics",
              "Grocery",
              "Clothing",
              "Grocery")
inventory$Category <- category
barplot(inventory$Quantity_Available,
        names.arg = inventory$Product_Name,
        col = c("skyblue", "lightgreen", "orange"),
        main = "Product Quantity by Category",
        xlab = "Product",
        ylab = "Quantity Available",
        legend.text = TRUE,
        args.legend = list(title = "Category",
                           x = "topright"))
```

Output:



3. In Tableau, build a dashboard combining the bar chart and stacked bar chart for interactive exploration of inventory data.

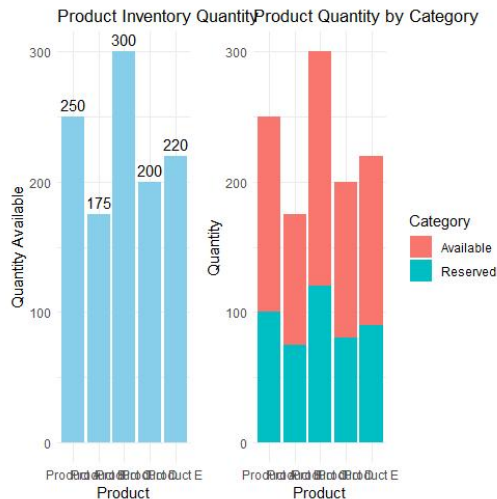
Code:

```

if (!require(ggplot2)) install.packages("ggplot2")
if (!require(patchwork)) install.packages("patchwork")
library(ggplot2)
library(patchwork)
product <- c("Product A", "Product B", "Product C",
             "Product D", "Product E")
quantity <- c(250, 175, 300, 200, 220)
category <- c("Electronics", "Electronics",
              "Grocery", "Clothing", "Grocery")
inventory <- data.frame(
  Product = product,
  Quantity = quantity,
  Category = category
)
bar_chart <- ggplot(inventory,
                    aes(x = Product, y = Quantity)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  geom_text(aes(label = Quantity),
            vjust = -0.5) +
  labs(
    title = "Product Inventory Quantity",
    x = "Product",
    y = "Quantity Available"
  ) +
  theme_minimal()
stacked_data <- data.frame(
  Product = rep(product, 2),
  Category = rep(c("Available", "Reserved"), each = 5),
  Quantity = c(
    150, 100, 180, 120, 130,
    100, 75, 120, 80, 90)
)
stacked_chart <- ggplot(stacked_data,
                        aes(x = Product,
                           y = Quantity,
                           fill = Category)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Product Quantity by Category",
    x = "Product",
    y = "Quantity",
    fill = "Category"
  ) +
  theme_minimal()
dashboard <- bar_chart + stacked_chart
dashboard

```

Output:

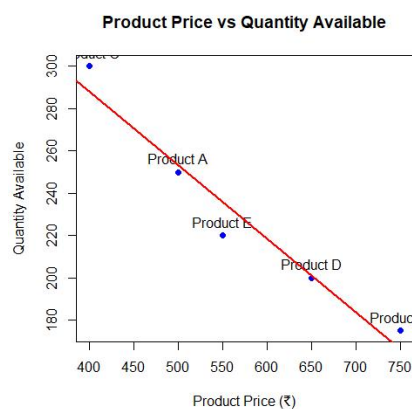


4. In R, create a scatter plot to explore the relationship between product price and quantity available. Explain any insights.

Code:

```
price <- c(500, 750, 400, 650, 550)
inventory$Price <- price
plot(inventory$Price,
     inventory$Quantity_Available,
     pch = 19,
     col = "blue",
     main = "Product Price vs Quantity Available",
     xlab = "Product Price (₹)",
     ylab = "Quantity Available")
text(inventory$Price,
     inventory$Quantity_Available,
     labels = inventory$Product_Name,
     pos = 3)
abline(lm(Quantity_Available ~ Price,
          data = inventory),
       col = "red",
       lwd = 2)
```

Output:



Set 25: Website Traffic Analysis

Dataset: Website Traffic

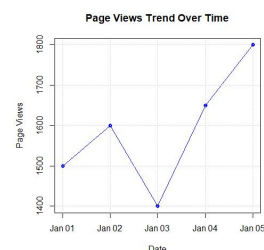
Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%
2023-01-04	1650	2.4%
2023-01-05	1800	2.6%

1. In Tableau, create a line chart to visualize the trend in page views over time. Label the axes and title the chart.

Code:

```
website <- data.frame(  
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03",  
                  "2023-01-04", "2023-01-05")),  
  Page_Views = c(1500, 1600, 1400, 1650, 1800),  
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)  
)  
plot(website$Date,  
     website$Page_Views,  
     type = "o",  
     pch = 16,  
     col = "blue",  
     main = "Page Views Trend Over Time",  
     xlab = "Date",  
     ylab = "Page Views")  
grid()
```

Output:



2. In R, generate a bar chart to show the top N days with the highest click-through rates. Label the chart elements.

Code:

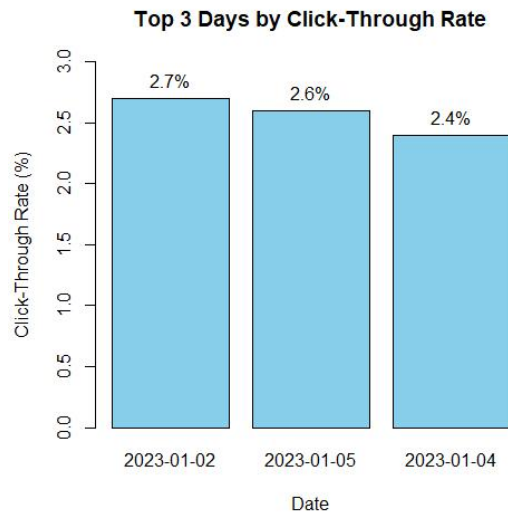
```
website <- data.frame(  
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03",  
                  "2023-01-04", "2023-01-05")),  
  Page_Views = c(1500, 1600, 1400, 1650, 1800),  
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)  
)  
sorted_data <- website[order(-website$CTR), ]  
top_n <- head(sorted_data, 3)  
bp <- barplot(top_n$CTR,  
              names.arg = format(top_n$Date, "%Y-%m-%d"),  
              col = "skyblue",  
              main = "Top 3 Days by Click-Through Rate",  
              xlab = "Date",
```

```

      ylab = "Click-Through Rate (%)",
      ylim = c(0, 3))
text(bp,
     top_n$CTR,
     labels = paste0(top_n$CTR, "%"),
     pos = 3)

```

Output:



3. In R, build a stacked area chart to display the distribution of user interactions (likes, shares, comments) on the website.

Code:

```

if (!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
interactions <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03",
                  "2023-01-04", "2023-01-05")),
  Likes = c(300, 350, 280, 370, 420),
  Shares = c(120, 150, 100, 160, 190),
  Comments = c(80, 90, 70, 100, 110)
)

interaction_long <- data.frame(
  Date = rep(interactions$Date, 3),
  Type = rep(c("Likes", "Shares", "Comments"), each = 5),
  Count = c(interactions$Likes,
            interactions$Shares,
            interactions$Comments)
)

ggplot(interaction_long,
      aes(x = Date,
          y = Count,
          fill = Type)) +
  geom_area(alpha = 0.8) +
  labs(
    title = "User Interactions Over Time",

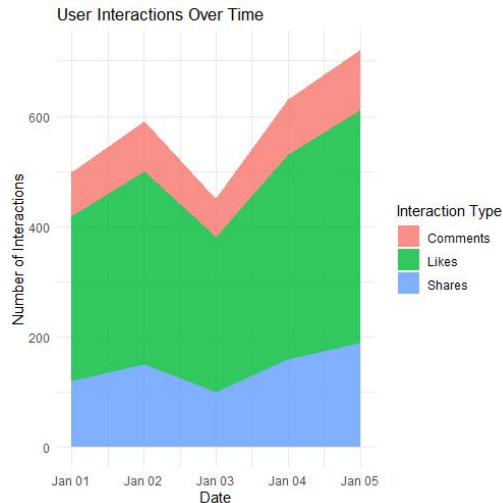
```

```

x = "Date",
y = "Number of Interactions",
fill = "Interaction Type"
) +
theme_minimal()

```

Output:



4. In Tableau, create a dashboard with an interactive map showing traffic sources and a bar chart displaying page views by source.

Code:

```

if (!require(shiny)) install.packages("shiny")
if (!require(leaflet)) install.packages("leaflet")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(leaflet)
library(ggplot2)
traffic_data <- data.frame(
  Source = c("Google", "Facebook", "Instagram",
            "YouTube", "Direct"),
  Page_Views = c(3200, 2400, 1800, 1500, 2100),
  Latitude = c(28.6139, 19.0760, 13.0827,
              12.9716, 17.3850),
  Longitude = c(77.2090, 72.8777, 80.2707,
                77.5946, 78.4867))
ui <- fluidPage(
  titlePanel("Website Traffic Analysis Dashboard"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        "source",
        "Select Traffic Source:",
        choices = c("All", traffic_data$Source),
        selected = "All" ),
    mainPanel(
      h3("Traffic Sources Map"),
      leafletOutput(

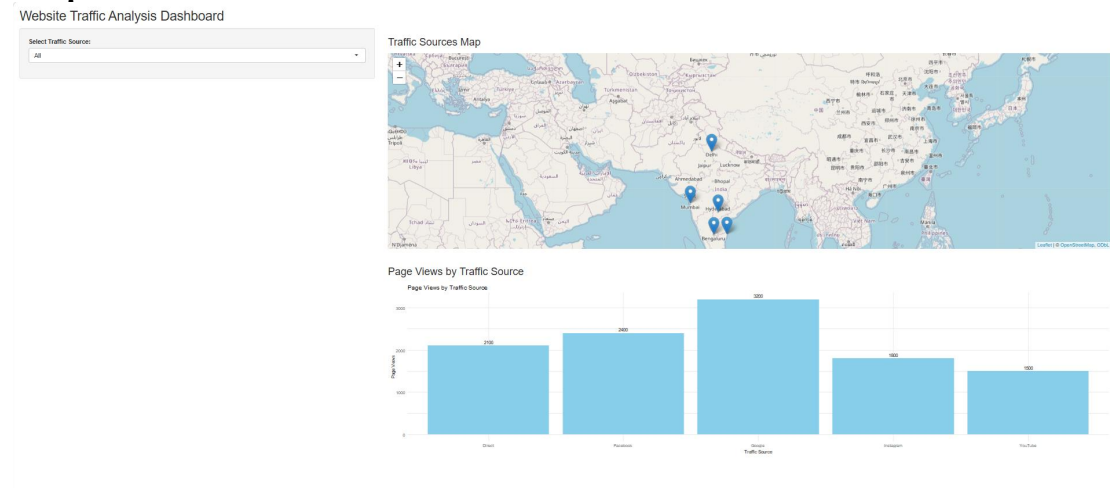
```

```

    "traffic_map",
    height = 450),
  br(),
  h3("Page Views by Traffic Source"),
  plotOutput(
    "pageviews_chart",
    height = 400))))
server <- function(input, output) {
  filtered_data <- reactive({
    if (input$source == "All") {
      traffic_data
    } else {
      traffic_data[
        traffic_data$Source == input$source, } } })
  output$traffic_map <- renderLeaflet({
    data <- filtered_data()
    leaflet(data) %>%
      addTiles() %>%
      addMarkers(
        lng = ~Longitude,
        lat = ~Latitude,
        popup = ~paste(
          "<b>Traffic Source:</b>", Source,
          "<br><b>Page Views:</b>", Page_Views ) ) })
  output$pageviews_chart <- renderPlot({
    data <- filtered_data()
    ggplot(
      data,
      aes(x = Source,
          y = Page_Views)
    ) +
      geom_bar(
        stat = "identity",
        fill = "skyblue"
      ) +
      geom_text(
        aes(label = Page_Views),
        vjust = -0.5
      ) +
      labs(
        title = "Page Views by Traffic Source",
        x = "Traffic Source",
        y = "Page Views"
      ) +
      theme_minimal() )})
shinyApp(
  ui = ui,
  server = server
)

```

Output:



SET 26

Dataset: Student_Mini_Data.csv

Student_ID	Gender	Study_Hours	Attendance	Math_Score	Science_Score	Exam_Date
S01	Male	2.0	78	62	65	2025-01-10
S02	Female	3.5	90	80	85	2025-01-10
S03	Male	1.5	70	55	58	2025-02-12
S04	Female	4.0	95	90	92	2025-02-12
S05	Male	2.8	85	72	74	2025-03-15
S06	Female	3.0	92	82	86	2025-03-15

1. Import the dataset in R. Plot a histogram for Math_Score. Draw a boxplot of Science_Score by

Gender. Add proper title and labels. Interpret the results.

Code:

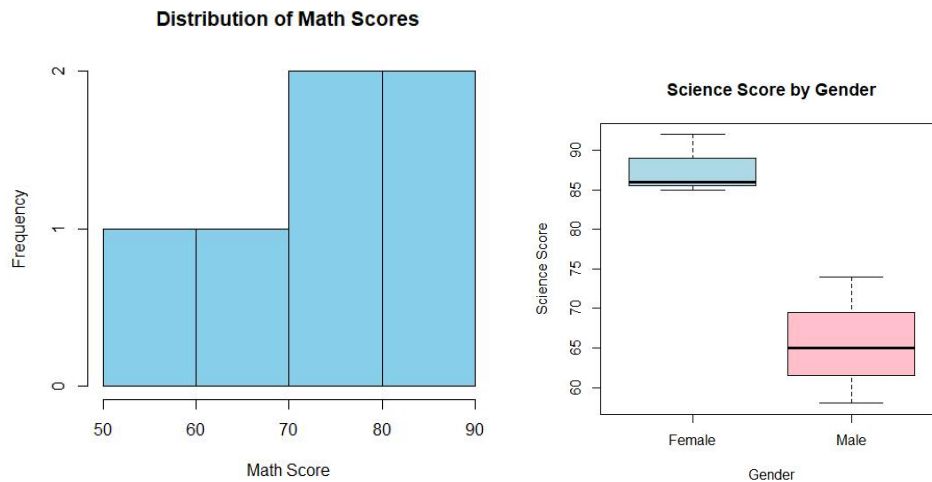
```
student <- data.frame(  
  Student_ID = c("S01", "S02", "S03", "S04", "S05", "S06"),  
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),  
  Study_Hours = c(2.0, 3.5, 1.5, 4.0, 2.8, 3.0),  
  Attendance = c(78, 90, 70, 95, 85, 92),  
  Math_Score = c(62, 80, 55, 90, 72, 82),  
  Science_Score = c(65, 85, 58, 92, 74, 86),  
  Exam_Date = c("2025-01-10", "2025-01-10",  
                "2025-02-12", "2025-02-12",  
                "2025-03-15", "2025-03-15")  
)  
student$Exam_Date <- as.Date(student$Exam_Date)  
print(student)  
hist(student$Math_Score,  
  main = "Distribution of Math Scores",  
  xlab = "Math Score",  
  ylab = "Frequency",  
  col = "skyblue",  
  border = "black")  
boxplot(Science_Score ~ Gender,  
  data = student,
```

```

main = "Science Score by Gender",
xlab = "Gender",
ylab = "Science Score",
col = c("lightblue", "pink"))

```

Output:



2. Create a scatter plot using R programming between Study_Hours and Math_Score. Use different colors for Gender. Add a regression line. Explain the relationship between study time and performance.

Code:

```

student <- data.frame(
  Student_ID = c("S01", "S02", "S03", "S04", "S05", "S06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Math_Score = c(62, 80, 55, 90, 72, 82),
  Exam_Date = c("2025-01-10", "2025-01-10",
    "2025-02-12", "2025-02-12",
    "2025-03-15", "2025-03-15")
)
student$Exam_Date <- as.Date(student$Exam_Date)

student$Month <- format(student$Exam_Date, "%Y-%m")
monthly_avg <- aggregate(
  Math_Score ~ Month,
  data = student,
  FUN = mean)
print(monthly_avg)
monthly_avg$Moving_Average <- c(
  NA,
  (monthly_avg$Math_Score[1] +
    monthly_avg$Math_Score[2]) / 2,
  (monthly_avg$Math_Score[2] +
    monthly_avg$Math_Score[3]) / 2)
monthly_avg$Month <- as.Date(
  paste0(monthly_avg$Month, "-01")
)

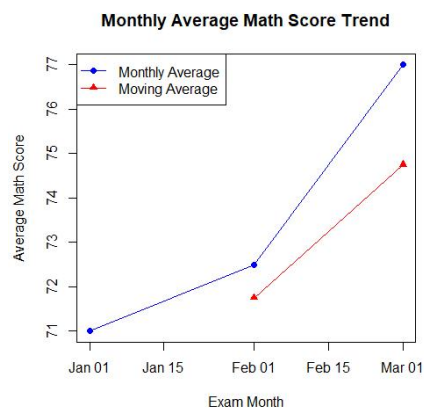
```

```

plot(monthly_avg$Month,
     monthly_avg$Math_Score,
     type = "o",
     pch = 16,
     col = "blue",
     main = "Monthly Average Math Score Trend",
     xlab = "Exam Month",
     ylab = "Average Math Score")
lines(monthly_avg$Month,
      monthly_avg$Moving_Average,
      type = "o",
      pch = 17,
      col = "red")
legend("topleft",
      legend = c("Monthly Average", "Moving Average"),
      col = c("blue", "red"),
      lty = 1,
      pch = c(16, 17))

```

Output:



3. Using R, Convert Exam_Date into Date format. Calculate monthly average Math scores. Plot a line chart for the trend. Apply moving average smoothing. Identify any increasing or decreasing pattern.

Code:

```

student <- data.frame(
  Student_ID = c("S01", "S02", "S03", "S04", "S05", "S06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Math_Score = c(62, 80, 55, 90, 72, 82),
  Exam_Date = c("2025-01-10", "2025-01-10",
                "2025-02-12", "2025-02-12",
                "2025-03-15", "2025-03-15"))
student$Exam_Date <- as.Date(student$Exam_Date)
student$Month <- format(student$Exam_Date, "%Y-%m")
monthly_avg <- aggregate(
  Math_Score ~ Month,
  data = student,
  FUN = mean)

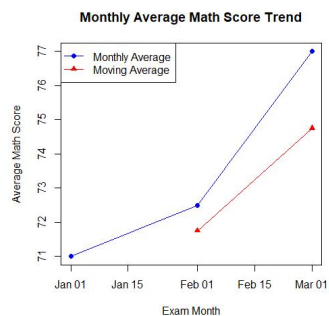
```

```

print(monthly_avg)
monthly_avg$Moving_Average <- c(
  NA,
  (monthly_avg$Math_Score[1] +
   monthly_avg$Math_Score[2]) / 2,
  (monthly_avg$Math_Score[2] +
   monthly_avg$Math_Score[3]) / 2)
monthly_avg$Month <- as.Date(
  paste0(monthly_avg$Month, "-01"))
plot(monthly_avg$Month,
     monthly_avg$Math_Score,
     type = "o",
     pch = 16,
     col = "blue",
     main = "Monthly Average Math Score Trend",
     xlab = "Exam Month",
     ylab = "Average Math Score")
lines(monthly_avg$Month,
     monthly_avg$Moving_Average,
     type = "o",
     pch = 17,
     col = "red")
legend("topleft",
     legend = c("Monthly Average", "Moving Average"),
     col = c("blue", "red"),
     lty = 1,
     pch = c(16, 17))

```

Output:



4. Import Student_Mini_Data.csv into Tableau. Create a bar chart showing average Math scores by Gender. Create a line chart for score trends over time. Add a filter for Exam_Date. Design a simple interactive dashboard.

Code:

```

if (!require(shiny)) install.packages("shiny")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(ggplot2)
student <- data.frame(
  Student_ID = c("S01", "S02", "S03", "S04", "S05", "S06"),
  Gender = c("Male", "Female", "Male",
             "Female", "Male", "Female"),

```

```

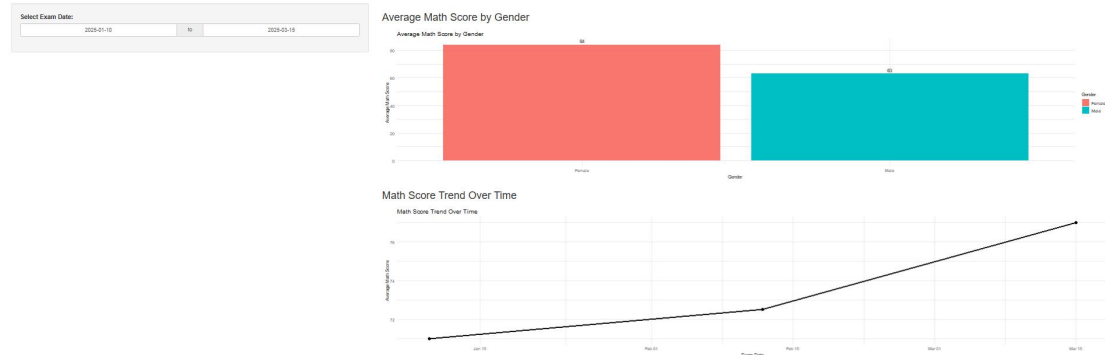
Study_Hours = c(2.0, 3.5, 1.5, 4.0, 2.8, 3.0),
Attendance = c(78, 90, 70, 95, 85, 92),
Math_Score = c(62, 80, 55, 90, 72, 82),
Science_Score = c(65, 85, 58, 92, 74, 86),
Exam_Date = as.Date(c(
  "2025-01-10",
  "2025-01-10",
  "2025-02-12",
  "2025-02-12",
  "2025-03-15",
  "2025-03-15"))
titlePanel("Student Performance Dashboard"),
sidebarLayout(
  sidebarPanel(
    dateRangeInput(
      "date",
      "Select Exam Date:",
      start = min(student$Exam_Date),
      end = max(student$Exam_Date),
      min = min(student$Exam_Date),
      max = max(student$Exam_Date)) ),
  mainPanel(
    h3("Average Math Score by Gender"),
    plotOutput(
      "gender_chart",
      height = "350px"
    ),
    h3("Math Score Trend Over Time"),

    plotOutput(
      "trend_chart",
      height = "350px"))))
server <- function(input, output) {
  filtered_data <- reactive({
    student[
      student$Exam_Date >= input$date[1] &
      student$Exam_Date <= input$date[2] ] })
  output$gender_chart <- renderPlot({
    data <- filtered_data()

```

Output:

Student Performance Dashboard



SET 27.

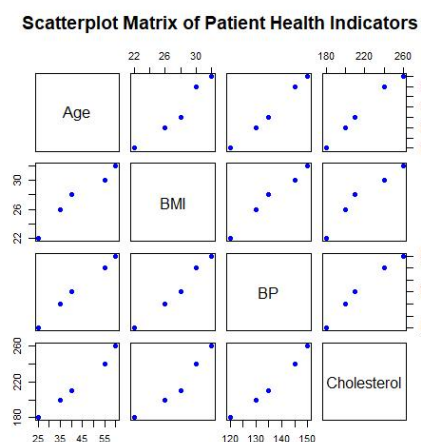
Patient Health Risk Analysis

Patient ID	Age	BMI	BP	Cholesterol
P1	25	22	120	180
P2	40	28	135	210
P3	55	30	145	240
P4	35	26	130	200
P5	60	32	150	260

1. Using R, create a scatterplot matrix for Age, BMI, BP, and Cholesterol. Examine pairwise relationships to identify strong correlations, clusters, or potential risk patterns.

Code:

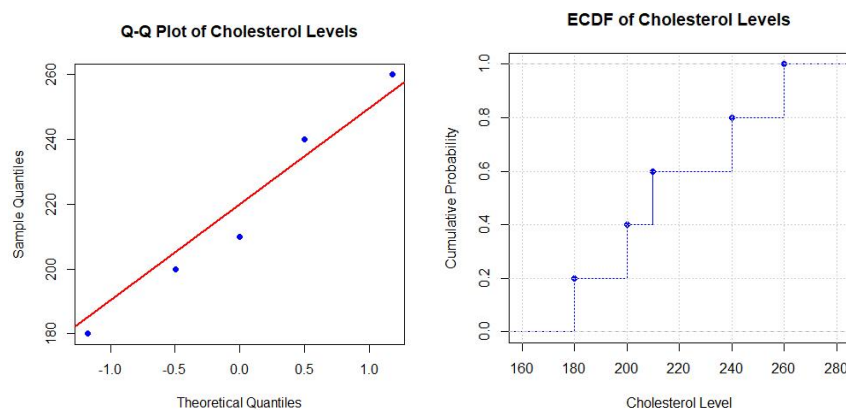
```
patient <- data.frame(
  Patient_ID = c("P1", "P2", "P3", "P4", "P5"),
  Age = c(25, 40, 55, 35, 60),
  BMI = c(22, 28, 30, 26, 32),
  BP = c(120, 135, 145, 130, 150),
  Cholesterol = c(180, 210, 240, 200, 260))
pairs(
  patient[, c("Age", "BMI", "BP", "Cholesterol")],
  main = "Scatterplot Matrix of Patient Health Indicators",
  pch = 19,
  col = "blue")
```



2. Construct Q-Q plot and ECDF for Cholesterol levels to assess distribution behaviour. Comment on normality assumption and skewness.

<pre>qqnorm(patient\$Cholesterol, main = "Q-Q Plot of Cholesterol Levels", pch = 19, col = "blue") qqline(patient\$Cholesterol, col = "red", lwd = 2)</pre>	<pre>plot(ecdf(patient\$Cholesterol), main = "ECDF of Cholesterol Levels", xlab = "Cholesterol Level", ylab = "Cumulative Probability", verticals = TRUE, do.points = TRUE, pch = 19, col = "blue")</pre>
---	---

Output:

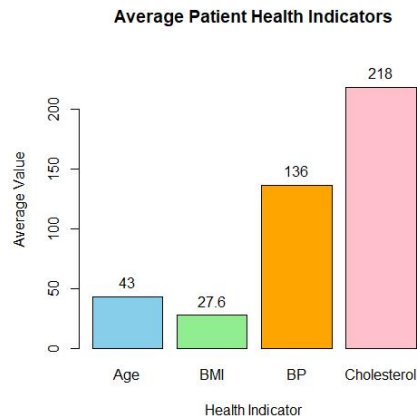


3. Using R, create a bar chart showing the average values of each health indicator (Age, BMI, BP, Cholesterol). Include axis labels, a descriptive title, and distinct colors for each bar. How do these averages relate to general patient health risk

Code:

```
averages <- c(
  Age = mean(patient$Age),
  BMI = mean(patient$BMI),
  BP = mean(patient$BP),
  Cholesterol = mean(patient$Cholesterol))
print(averages)
bar_positions <- barplot(
  averages,
  col = c("skyblue", "lightgreen", "orange", "pink"),
  main = "Average Patient Health Indicators",
  xlab = "Health Indicator",
  ylab = "Average Value",
  ylim = c(0, max(averages) + 30))
text(
  x = bar_positions,
  y = averages,
  labels = round(averages, 1),
  pos = 3)
```

Output:



4. Create an interactive Tableau dashboard with a heatmap highlighting high-risk patients and an Age filter for exploring risk patterns. Include hover tooltips showing all health indicators. Analyze which patients are highest risk, how age affects risk, and how this dashboard can aid healthcare professionals in prioritizing care.

Code:

```
if (!require(shiny)) install.packages("shiny")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(ggplot2)
patient <- data.frame(
  Patient_ID = c("P1", "P2", "P3", "P4", "P5"),
  Age = c(25, 40, 55, 35, 60),
  BMI = c(22, 28, 30, 26, 32),
  BP = c(120, 135, 145, 130, 150),
  Cholesterol = c(180, 210, 240, 200, 260)
)
patient$Risk_Score <-
  (patient$BMI >= 25) +
  (patient$BP >= 130) +
  (patient$Cholesterol >= 200)
patient$Risk_Level <- ifelse(
  patient$Risk_Score >= 3,
  "High Risk",
  ifelse(
    patient$Risk_Score == 2,
    "Moderate Risk",
    "Low Risk"))
ui <- fluidPage(
  titlePanel("Patient Health Risk Analysis Dashboard"),
  sidebarLayout(
    sidebarPanel(
      sliderInput(
        "age",
        "Select Maximum Age:",
        min = min(patient$Age),
        max = max(patient$Age),
        value = max(patient$Age),
```

```

    step = 1) ),
  mainPanel(
    h3("Patient Health Risk Heatmap"),
    plotOutput(
      "heatmap",
      height = "500px ),
    h4("Risk Analysis"),
    tableOutput("risk_table")
  )))server <- function(input, output) {
  filtered_data <- reactive({
    patient[
      patient$Age <= input$age,
    ]
  })
  output$heatmap <- renderPlot({
    data <- filtered_data()
    ggplot(
      data,
      aes(
        x = Patient_ID,
        y = Risk_Score,
        fill = Risk_Level
      )
    ) +
    geom_tile(
      color = "white",
      height = 0.8
    ) +
    geom_text(
      aes(
        label = paste(
          "Age:", Age,
          "\nBMI:", BMI,
          "\nBP:", BP,
          "\nChol:", Cholesterol
        )
      ),
      size = 4
    ) +
    labs(
      title = "Patient Risk Heatmap",
      x = "Patient",
      y = "Risk Score",
      fill = "Risk Level"
    ) +
    theme_minimal()
  })
  output$risk_table <- renderTable({
    data <- filtered_data()
    data.frame(
      Patient_ID = data$Patient_ID,

```

```

Age = data$Age,
BMI = data$BMI,
BP = data$BP,
Cholesterol = data$Cholesterol,
Risk_Level = data$Risk_Level)))))
shinyApp(
  ui = ui,
  server = server)
Output:

```



SET 28. Vehicle Performance Analysis

Dataset:

Vehicle_ID	Engine_Size(L)	Horsepower	Fuel_Efficiency(km/l)	Top_Speed(km/h)	Safety_Rating
V1	1.5	110	18	180	4
V2	2.0	150	15	200	5
V3	3.0	250	12	250	5
V4	2.5	200	14	220	4
V5	1.8	130	17	190	3

1. Using R, develop a violin plot to compare the distribution of Fuel_Efficiency across different Safety_Rating categories. Examine the spread, central tendency, and variability

within each group, and interpret what this indicates about efficiency patterns.

Code:

```

if(!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
vehicle <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)
vehicle$Safety_Rating <- as.factor(vehicle$Safety_Rating)
ggplot(
  vehicle,

```

```

aes(
  x = Safety_Rating,
  y = Fuel_Efficiency,
  fill = Safety_Rating
)
) +
geom_violin(
  trim = FALSE,
  alpha = 0.7
) +

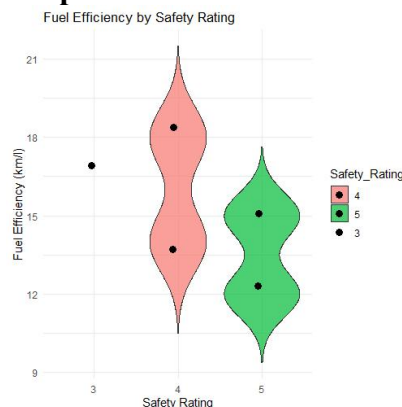
geom_jitter(
  width = 0.08,
  size = 3
) +

labs(
  title = "Fuel Efficiency by Safety Rating",
  x = "Safety Rating",
  y = "Fuel Efficiency (km/l)"
) +

theme_minimal()

```

Output:



2. Using R, construct a scatter plot to explore the relationship between Horsepower and Top_Speed, applying color differentiation based on Engine_Size. Analyze the strength and direction of correlation and find observable performance trends.

Code:

```

if (!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
vehicle <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)

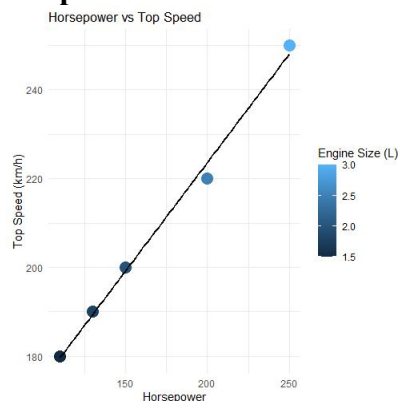
```

```

ggplot(
  vehicle,
  aes(
    x = Horsepower,
    y = Top_Speed,
    color = Engine_Size
  )
) +
  geom_point(
    size = 5
  ) +
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "black"
  ) +
  labs(
    title = "Horsepower vs Top Speed",
    x = "Horsepower",
    y = "Top Speed (km/h)",
    color = "Engine Size (L)"
  ) +
  theme_minimal()

```

Output:



3. Using R, create a correlation heatmap for all numerical variables in the dataset. Identify the variables most strongly associated with Top_Speed and their influence on overall vehicle performance.

Code:

```

if (!require(ggplot2)) install.packages("ggplot2")
if (!require(reshape2)) install.packages("reshape2")
library(ggplot2)
library(reshape2)
vehicle <- data.frame(
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)

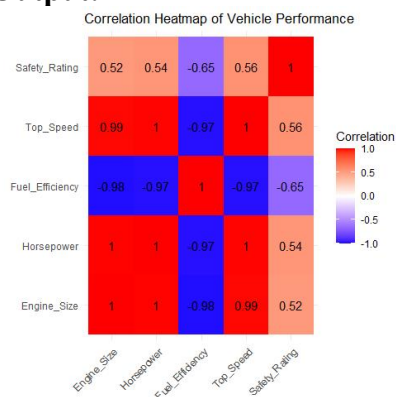
```

```

cor_matrix <- cor(vehicle)
print(round(cor_matrix, 2))
cor_data <- melt(cor_matrix)
ggplot(
  cor_data,
  aes(
    x = Var1,
    y = Var2,
    fill = value
  )
) +
  geom_tile() + geom_text(
    aes(label = round(value, 2)),
    size = 4
  ) +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0,
    limits = c(-1, 1)
  ) +
  labs(
    title = "Correlation Heatmap of Vehicle Performance",
    x = "",
    y = "",
    fill = "Correlation"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
)

```

Output:



4. In Tableau, design an interactive dashboard incorporating bar charts and scatter plots, along with filters for Engine_Size and Safety_Rating, to highlight vehicles that balance fuel efficiency and high performance. Provide analytical interpretation of the insights derived from the dashboard.

Code:

```

if (!require(shiny)) install.packages("shiny")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(ggplot2)
vehicle <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)
ui <- fluidPage(
  titlePanel("Vehicle Performance Analysis Dashboard"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        "engine",
        "Select Engine Size:",
        choices = c(
          "All",
          sort(unique(vehicle$Engine_Size))
        ),
        selected = "All"
      ),
      selectInput(
        "safety",
        "Select Safety Rating:",
        choices = c(
          "All",
          sort(unique(vehicle$Safety_Rating))
        ),
        selected = "All"
      ),
      plotOutput(
        "bar_chart",
        height = "350px"
      ),
      h3("Horsepower vs Top Speed"),

      plotOutput(
        "scatter_chart",
        height = "400px"
      ),
      h3("Selected Vehicle Data"),
      tableOutput("vehicle_table"))))
server <- function(input, output) {
  filtered_data <- reactive({
    data <- vehicle
    if (input$engine != "All") {
      data <- data[
        data$Engine_Size ==

```

```

        as.numeric(input$engine),
      ]}
    if (input$safety != "All") {
      data <- data[
        data$Safety_Rating ==
          as.numeric(input$safety),
      ]
    }
    data
  })
  output$bar_chart <- renderPlot({
    data <- filtered_data(
      ggplot(
        data,
        aes(
          x = Vehicle_ID,
          y = Fuel_Efficiency
        )
      ) +
      geom_bar(
        stat = "identity",
        fill = "skyblue"
      ) +
      geom_text(
        aes(
          label = Fuel_Efficiency
        ),
        vjust = -0.5
      ) +
      labs(
        title = "Fuel Efficiency by Vehicle",
        x = "Vehicle",
        y = "Fuel Efficiency (km/l)"
      ) +
      theme_minimal()
    })
  output$scatter_chart <- renderPlot({
    data <- filtered_data()
    ggplot(
      data,
      aes(
        x = Horsepower,
        y = Top_Speed,
        color = Engine_Size
      )
    ) +
    geom_point(
      size = 5
    ) +
    labs(

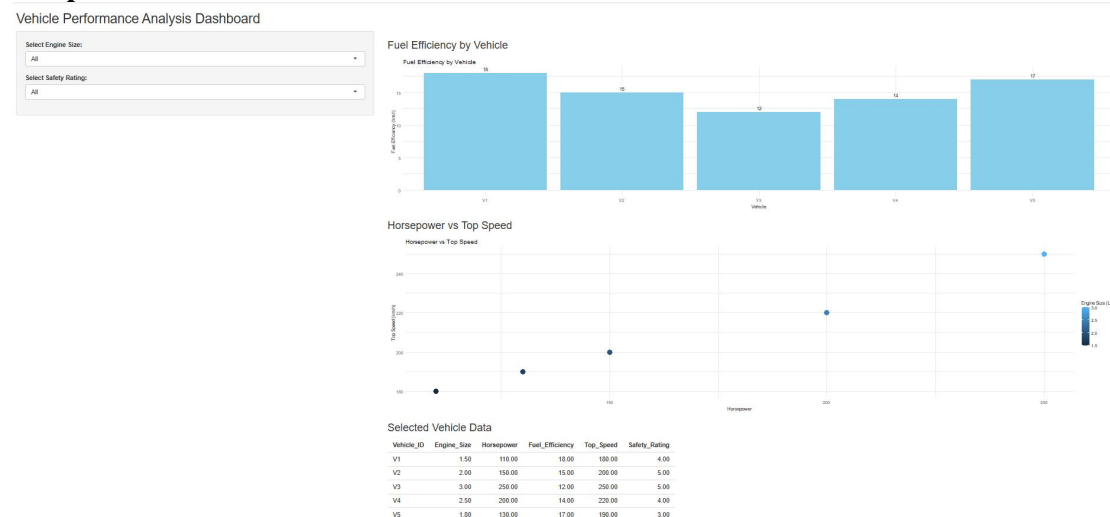
```

```

    title = "Horsepower vs Top Speed",
    x = "Horsepower",
    y = "Top Speed (km/h)",
    color = "Engine Size (L)"
  ) +
  theme_minimal()
})
output$vehicle_table <- renderTable({
  filtered_data()
})
}
shinyApp(
  ui = ui,
  server = server
)

```

Output:



SET 29 .Student Academic Performance

Dataset:

Student_ID	Age	Study_Hours	Attendance(%)	Test_Score	Participation_Score
S1	19	12	90	85	8
S2	21	8	70	70	7
S3	20	15	95	92	9
S4	22	10	85	80	8
S5	23	7	60	65	6

1. Using R, create a stacked area chart showing Test_Score and Participation_Score across students. Analyze trends.

Code:

```

if(!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
student <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),
  Attendance = c(90, 70, 95, 85, 60),
  Test_Score = c(85, 70, 92, 80, 65),

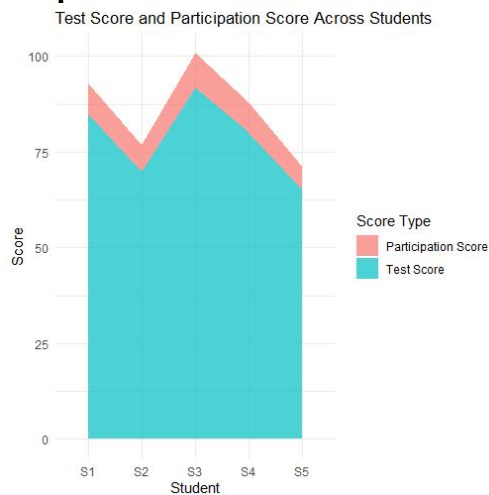
```

```

    Participation_Score = c(8, 7, 9, 8, 6)
  )
  student_long <- data.frame(
    Student_ID = rep(student$Student_ID, 2),
    Score_Type = rep(
      c("Test Score", "Participation Score"),
      each = 5
    ),
    Score = c(
      student$Test_Score,
      student$Participation_Score))
  ggplot(
    student_long,
    aes(
      x = Student_ID,
      y = Score,
      fill = Score_Type,
      group = Score_Type
    )
  ) + geom_area(alpha = 0.7) +
  labs(
    title = "Test Score and Participation Score Across Students",
    x = "Student",
    y = "Score",
    fill = "Score Type"
  ) +
  theme_minimal()

```

Output:



2. Using R, generate a boxplot of Study_Hours grouped by Attendance quartiles. Interpret patterns. Include axis labels, title, and colors for each quartile. Analyze the boxplot to identify median and interquartile range of study hours per attendance quartile and any outliers in study hours.

Code:

```

student <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),

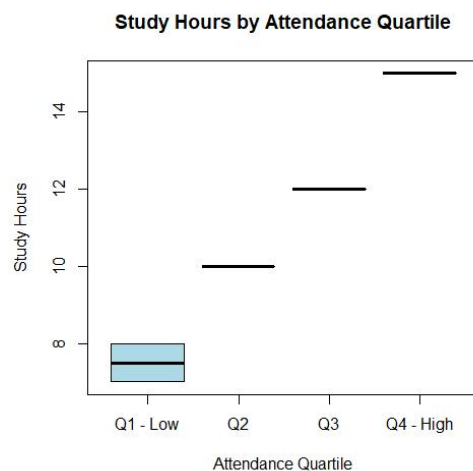
```

```

Attendance = c(90, 70, 95, 85, 60),
Test_Score = c(85, 70, 92, 80, 65),
Participation_Score = c(8, 7, 9, 8, 6)
)
student$Attendance_Quartile <- cut(
  student$Attendance,
  breaks = quantile(
    student$Attendance,
    probs = c(0, 0.25, 0.50, 0.75, 1)
  ),
  include.lowest = TRUE,
  labels = c(
    "Q1 - Low",
    "Q2",
    "Q3",
    "Q4 - High"
  )
)
)
boxplot(
  Study_Hours ~ Attendance_Quartile,
  data = student,
  col = c(
    "lightblue",
    "lightgreen",
    "orange",
    "pink"
  ),
  main = "Study Hours by Attendance Quartile",
  xlab = "Attendance Quartile",
  ylab = "Study Hours"
)

```

Output:



3. Using R, construct a density plot for Test_Score to evaluate score distribution and outliers.

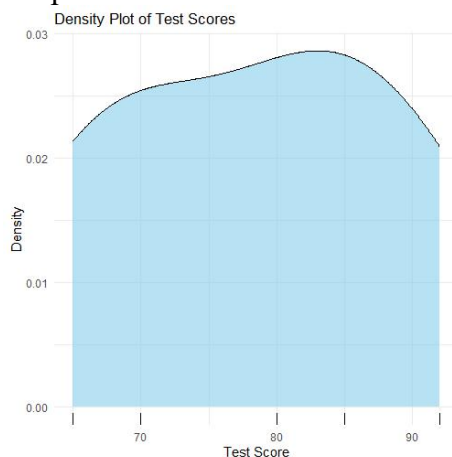
Code:

```

if (!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
student <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Test_Score = c(85, 70, 92, 80, 65)
)
ggplot(
  student,
  aes(x = Test_Score)) +
  geom_density(
    fill = "skyblue",
    alpha = 0.6 ) +
  geom_rug(
    sides = "b" ) +
  labs(
    title = "Density Plot of Test Scores",
    x = "Test Score",
    y = "Density"
  ) +
  theme_minimal()

```

Output:



4. In Tableau, design an interactive scatterplot with Study_Hours on X-axis, Test_Score on Y-axis, colored by Participation_Score. Include filters for Age.

Code:

```

if (!require(shiny)) install.packages("shiny")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(ggplot2)
student <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),
  Attendance = c(90, 70, 95, 85, 60),
  Test_Score = c(85, 70, 92, 80, 65),
  Participation_Score = c(8, 7, 9, 8, 6))
ui <- fluidPage(
  titlePanel("Student Academic Performance Dashboard"),

```

```

sidebarLayout(
  sidebarPanel(
    checkboxGroupInput(
      "age",
      "Select Age:",
      choices = sort(unique(student$Age)),
      selected = sort(unique(student$Age)) ),
    mainPanel(
      h3("Study Hours vs Test Score"),
      plotOutput(
        "scatter",
        height = "500px" ),
      h3("Selected Student Data"),
      tableOutput("student_table"))))
server <- function(input, output) {
  filtered_data <- reactive({
    student[
      student$Age %in% input$age,
    ]})
  output$scatter <- renderPlot({

    data <- filtered_data()
    ggplot(
      data,
      aes(
        x = Study_Hours,
        y = Test_Score,
        color = Participation_Score) ) +
    geom_point(
      size = 6) +
    geom_text(
      aes(label = Student_ID),
      vjust = -1,
      size = 4 ) +
    scale_color_gradient(
      low = "blue",
      high = "red" ) +
    labs(
      title = "Study Hours vs Test Score",
      x = "Study Hours",
      y = "Test Score",
      color = "Participation Score" ) +
    theme_minimal() })
  output$student_table <- renderTable({
    filtered_data() })}
shinyApp(
  ui = ui,
  server = server
)

```

Output:

Student Academic Performance Dashboard

Select Age:

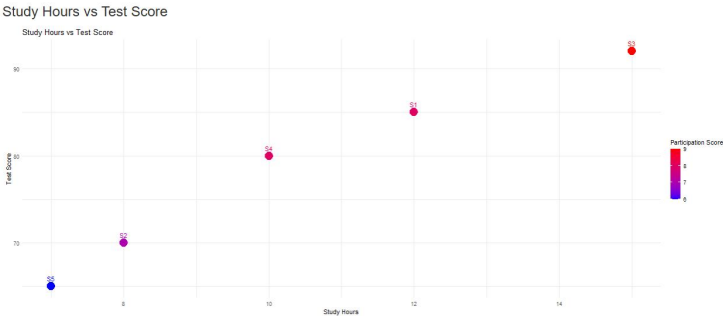
☒ 19

☒ 20

☒ 21

☒ 22

☒ 23



Selected Student Data

Student_ID	Age	Study_Hours	Attendance	Test_Score	Participation_Score
S1	19.00	12.00	90.00	85.00	8.00
S2	21.00	8.00	70.00	70.00	7.00
S3	20.00	15.00	95.00	92.00	9.00
S4	22.00	10.00	85.00	80.00	8.00
S5	23.00	7.00	60.00	65.00	6.00

SET 30

Dataset: Mobile_App_Usage.csv						
User_ID	Gender	Age	Screen_Time	(hrs)	App_Usage_Count	Data_Used
(GB) Satisfaction Usage_Date						
U01	Male	20	4.5	18	2.4	3
U02	Female	22	6.0	25	3.8	5
U03	Male	19	3.2	12	1.6	3
U04	Female	21	7.1	30	4.5	5
U05	Male	23	2.8	10	1.2	2
U06	Female	20	5.4	22	3.1	4

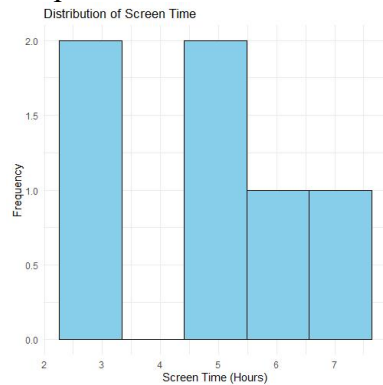
1. Import the dataset in R. Plot a histogram and density plot for Screen_Time. Add suitable colors and labels. Interpret the screen-time distribution.

Code:

```
if(!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male",
             "Female", "Male", "Female"),
  Age = c(20, 22, 19, 21, 23, 20),
  Screen_Time = c(4.5, 6.0, 3.2, 7.1, 2.8, 5.4),
  App_Usage_Count = c(18, 25, 12, 30, 10, 22),
  Data_Used = c(2.4, 3.8, 1.6, 4.5, 1.2, 3.1),
  Satisfaction = c(3, 5, 3, 5, 2, 4),
  Usage_Date = as.Date(c(
    "2025-01-08",
    "2025-01-08",
    "2025-02-11",
    "2025-02-11",
    "2025-03-14",
    "2025-03-14")))
ggplot(
  mobile,
  aes(x = Screen_Time)
) +
```

```
geom_histogram(
  bins = 5,
  fill = "skyblue",
  color = "black" ) +
labs(
  title = "Distribution of Screen Time",
  x = "Screen Time (Hours)",
  y = "Frequency" ) +
theme_minimal()
```

Output:



2. Create a scatter plot using R programming between Data_Used and Screen_Time. Calculate the correlation between them. Add a trend line. Explain the relationship.

Code:

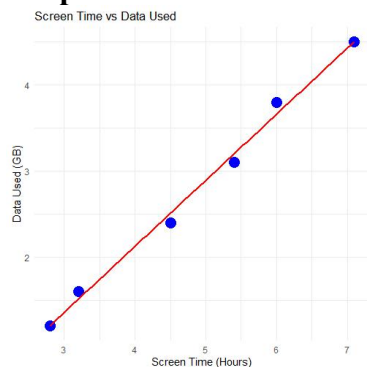
```
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Screen_Time = c(4.5, 6.0, 3.2, 7.1, 2.8, 5.4),
  Data_Used = c(2.4, 3.8, 1.6, 4.5, 1.2, 3.1))
correlation <- cor(
  mobile$Data_Used,
  mobile$Screen_Time)
print(
  paste(
    "Correlation =", round(correlation, 3)))
ggplot(
  mobile,
  aes(
    x = Screen_Time,
    y = Data_Used
  )
) +
geom_point(
  size = 5,
  color = "blue"
) +
geom_smooth(
  method = "lm",
  se = FALSE,
  color = "red" ) +
labs(
```

```

title = "Screen Time vs Data Used",
x = "Screen Time (Hours)",
y = "Data Used (GB)" ) +
theme_minimal()

```

Output:



3. Using R, find the average Satisfaction score for each Gender. Create a bar chart for the averages. Add value labels. Interpret user satisfaction patterns.

Code:

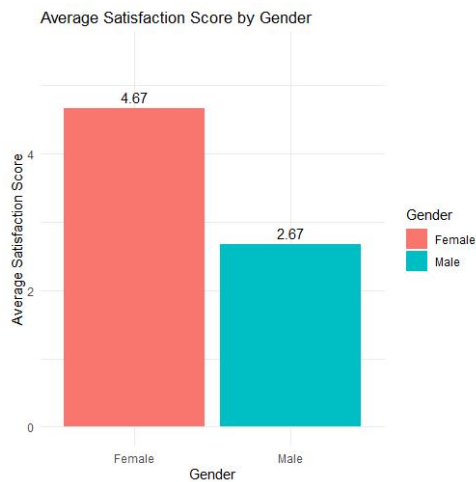
```

if (!require(ggplot2)) install.packages("ggplot2")

library(ggplot2)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male",
             "Female", "Male", "Female"),
  Satisfaction = c(3, 5, 3, 5, 2, 4))
avg_satisfaction <- aggregate(
  Satisfaction ~ Gender,
  data = mobile,
  FUN = mean)
print(avg_satisfaction)
ggplot(
  avg_satisfaction,
  aes(
    x = Gender,
    y = Satisfaction,
    fill = Gender )) +
  geom_bar(
    stat = "identity" ) +
  geom_text(
    aes(
      label = round(Satisfaction, 2)),
    vjust = -0.5 ) +
  labs(
    title = "Average Satisfaction Score by Gender",
    x = "Gender",
    y = "Average Satisfaction Score" ) +
  ylim(
    0,

```

```
5.5 ) +
theme_minimal()
Output:
```



4. Import Mobile_App_Usage.csv into Tableau. Create a bubble chart (Screen Time vs Data Used). Use color for Gender and size for App Usage Count. Create a line chart for usage trends. Add filters for Gender and Usage_Date. Design a mini dashboard.

Code:

```
if (!require(shiny)) install.packages("shiny")
if (!require(ggplot2)) install.packages("ggplot2")
library(shiny)
library(ggplot2)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03",
              "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male",
             "Female", "Male", "Female"),
  Age = c(20, 22, 19, 21, 23, 20),
  Screen_Time = c(4.5, 6.0, 3.2,
                  7.1, 2.8, 5.4),
  App_Usage_Count = c(18, 25, 12,
                      30, 10, 22),
  Data_Used = c(2.4, 3.8, 1.6,
                 4.5, 1.2, 3.1),
  Satisfaction = c(3, 5, 3,
                   5, 2, 4),
  Usage_Date = as.Date(c(
    "2025-01-08",
    "2025-01-08",
    "2025-02-11",
    "2025-02-11",
    "2025-03-14",
    "2025-03-14" )))
ui <- fluidPage(
  titlePanel(
    "Mobile App Usage Mini Dashboard" ),
```

```

sidebarLayout(
  sidebarPanel(
    checkboxGroupInput(
      "gender",
      "Select Gender:",
      choices = unique(mobile$Gender),
      selected = unique(mobile$Gender) ),
    dateRangeInput(
      "date",
      "Select Usage Date:",
      start = min(mobile$Usage_Date),
      end = max(mobile$Usage_Date),
      min = min(mobile$Usage_Date),
      max = max(mobile$Usage_Date)))
  mainPanel(
    h3("Screen Time vs Data Used"),
    plotOutput(
      "bubble_chart",
      height = "400px")
    h3("Usage Trend Over Time"),
    plotOutput(
      "line_chart",
      height = "400px"),
    h3("Selected User Data"),
    tableOutput(
      "user_table"))))
server <- function(input, output) {
  filtered_data <- reactive({
    mobile[
      mobile$Gender %in% input$gender &
      mobile$Usage_Date >= input$date[1] &
      mobile$Usage_Date <= input$date[2],
    ] })
  output$bubble_chart <- renderPlot({
    data <- filtered_data()
    ggplot(
      data,
      aes(
        x = Screen_Time,
        y = Data_Used,
        color = Gender,
        size = App_Usage_Count))+
      geom_point(
        alpha = 0.7 ) +
    geom_text(
      aes(label = User_ID),
      vjust = -1,
      size = 4 ) +
    labs(
      title = "Screen Time vs Data Used",

```

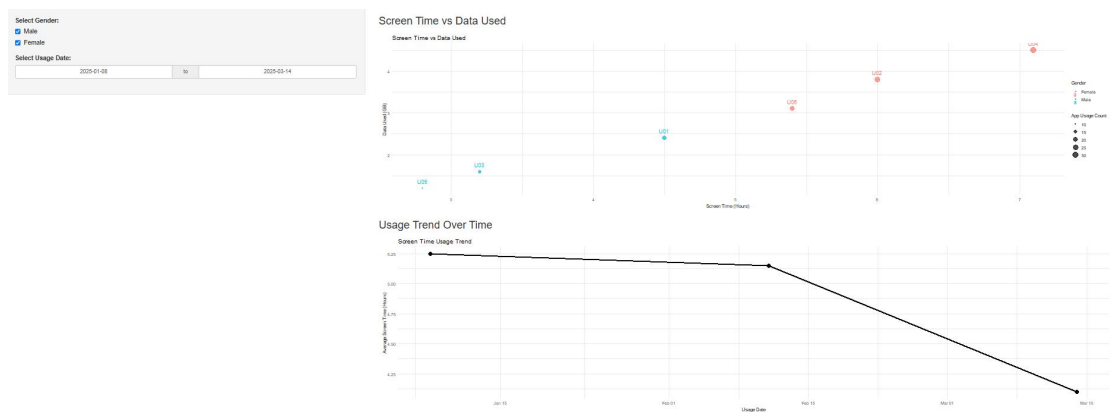
```

    x = "Screen Time (Hours)",
    y = "Data Used (GB)",
    color = "Gender",
    size = "App Usage Count" ) +
    theme_minimal() })
output$line_chart <- renderPlot({
  data <- filtered_data()
  daily_data <- aggregate(
    Screen_Time ~ Usage_Date,
    data = data,
    FUN = mean)
  ggplot(
    daily_data,
    aes(
      x = Usage_Date,
      y = Screen_Time)) +
    geom_line(
      linewidth = 1
    ) +

    geom_point(
      size = 4
    ) +
    labs(
      title = "Screen Time Usage Trend",
      x = "Usage Date",
      y = "Average Screen Time (Hours)"
    ) +
    theme_minimal())
output$user_table <- renderTable({
  filtered_data()})
shinyApp(
  ui = ui,
  server = server
)

```

Output:



Selected User Data

User_ID	Gender	Age	Screen_Time	App_Usage_Count	Data_Used	Satisfaction	Usage_Date
U01	Male	20.00	4.50	18.00	2.40	3.00	20096.00
U02	Female	22.00	6.00	25.00	3.80	5.00	20096.00
U03	Male	19.00	3.20	12.00	1.60	3.00	20130.00
U04	Female	21.00	7.10	30.00	4.50	5.00	20130.00
U05	Male	23.00	2.80	10.00	1.20	2.00	20161.00
U06	Female	20.00	5.40	22.00	3.10	4.00	20161.00